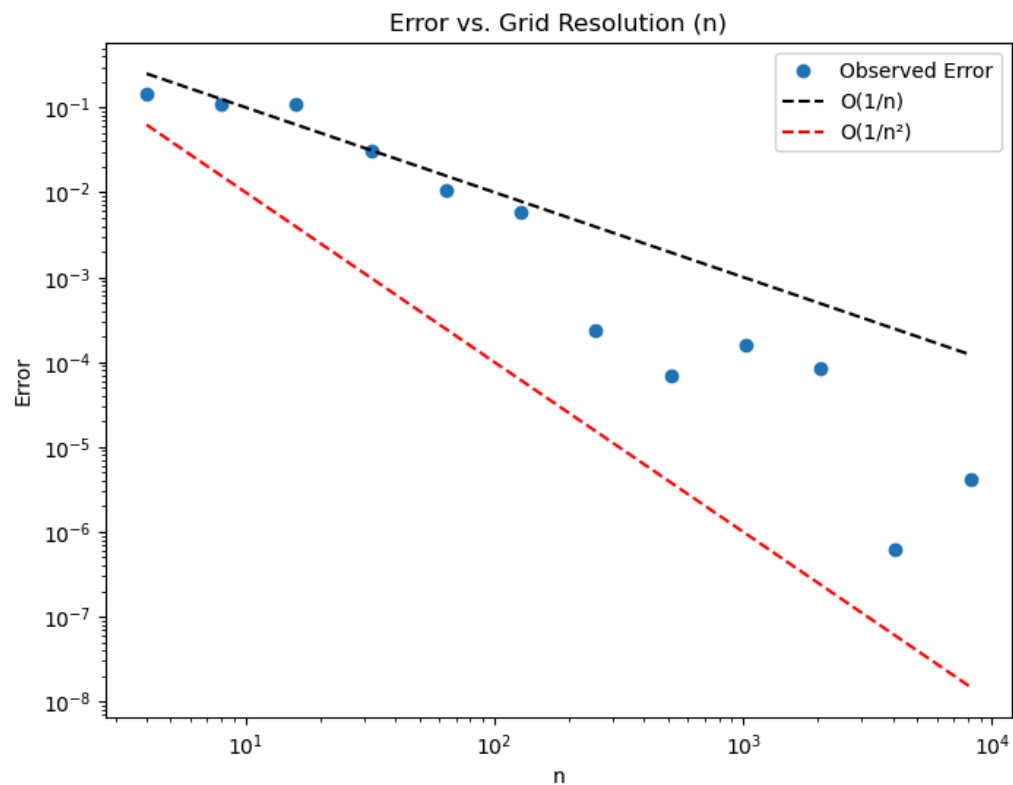


CO21BTECH11002
Aayush Kumar
ME5470 HW2 Report

Q1)

a)

n	error
4	1.415926535898e-01
8	1.084073464102e-01
16	1.084073464102e-01
32	3.028234641021e-02
64	1.075109641021e-02
128	5.868283910207e-03
256	2.352317147931e-04
512	6.994406645688e-05
1024	1.614968008319e-04
2048	8.520285551938e-05
4098	6.278329571785e-07
8192	4.140538624853e-06



Order of Accuracy:

From the graph, the blue dots align closely with the black dashed line ($O(1/n)$) initially, but at larger n , the dots are between $O(1/n)$ and $O(1/n^2)$.

Hence, for the used values of n , the order of accuracy is between $O(1/n)$ and $O(1/n^2)$. Though, for larger values of n , the order of accuracy would be $O(1/n^2)$.

b)

Variables	Shared/Private	Reason
i, j	Private	Each thread needs its own loop indices.
dist_sq	Private	Computed independently for each grid point.
xg, yg	Shared	Read by all the threads.
xcen, ycen, rad_sq, da	Shared	Constants used by all threads.
area_cir	Shared (Reduction)	Each thread accumulates a sum.
flag[i][j]	Shared	Each thread modifies a different flag[i][j], no conflicts.

c)

p	Error
2	4.140538624853e-06
4	4.140538624853e-06
8	4.140538624853e-06

The error remains the same for different numbers of threads because the computation is deterministic and does not depend on the order of execution.

Q2)

Loops Amenable to Parallelization:

1. `calc_yhat` Function:
 - a. Iterating over m data points to compute $yhat[i]$. Each iteration is independent, making it suitable for parallelization.
2. `calc_new_theta` Function:
 - a. Summation of gradients over m data points for each parameter $theta[j]$. The inner loop is independent for each i , allowing reduction(`+:gradJ`) to accumulate the sum safely.
3. `calc_Jfun` Function:
 - a. Computing sum of squared errors over m data points. Each iteration updates $Jfun$ independently, making reduction(`+:Jfun`) necessary to avoid race conditions.
4. `scale_and_translate_data` Function:
 - a. Computing $means[j]$ and $sdevs[j]$ for each feature. Loop operates independently for different i and j , allowing parallel execution.

- b. Summing feature values over m data points (reduction used for safe accumulation). Loop operates independently for different i and j, allowing parallel execution.
 - c. Normalizing each data point independently. Loop operates independently for different i and j, allowing parallel execution.
5. `undo_scale_and_translate_data` Function:
- a. Scaling and translating data back to original form over m data points. Each 'i' iteration is independent.

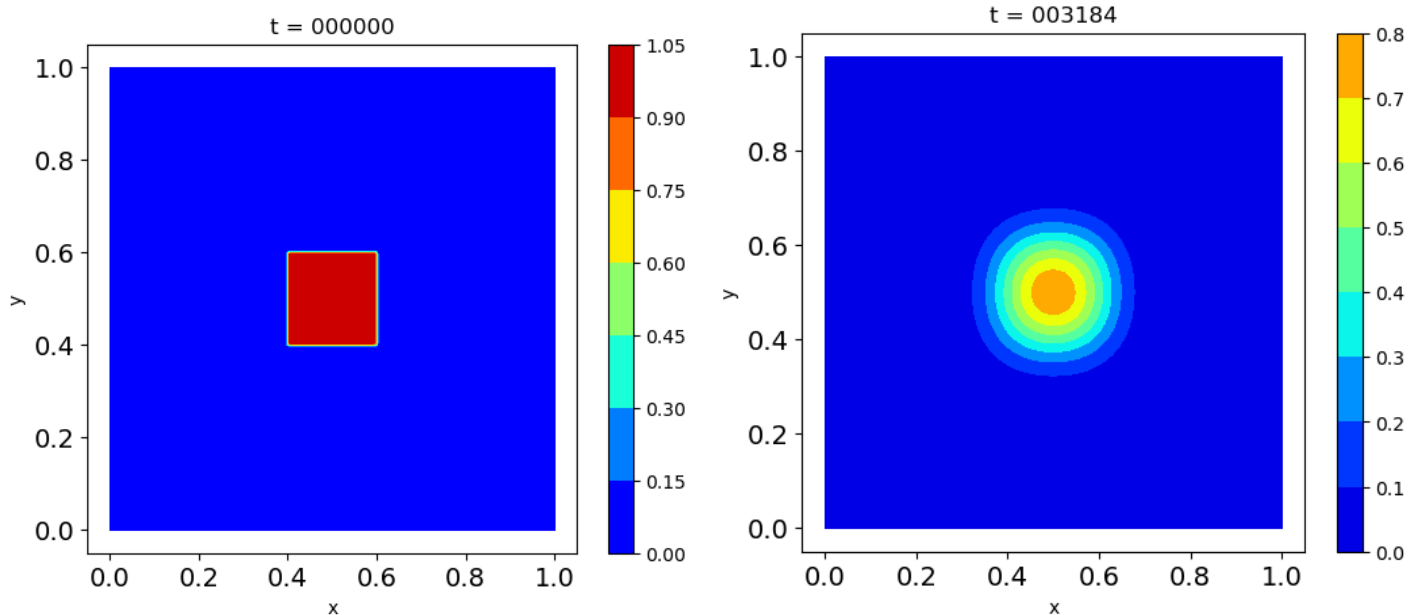
Final loss function values (at iter 28):

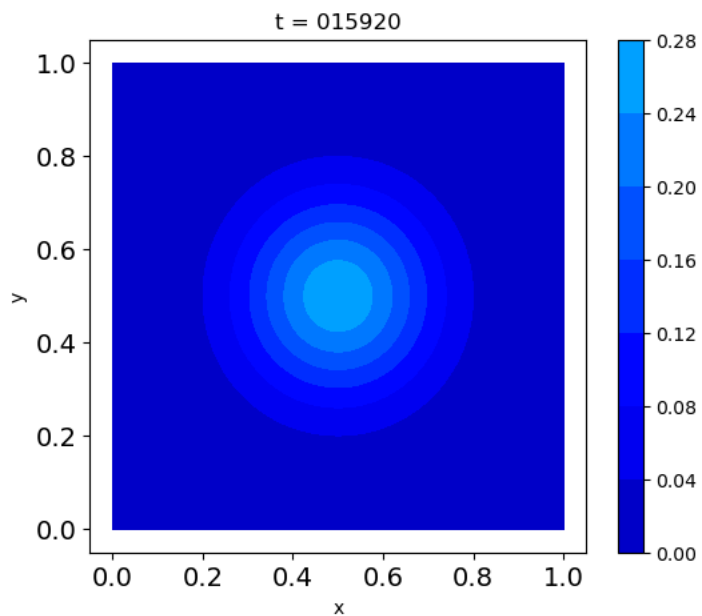
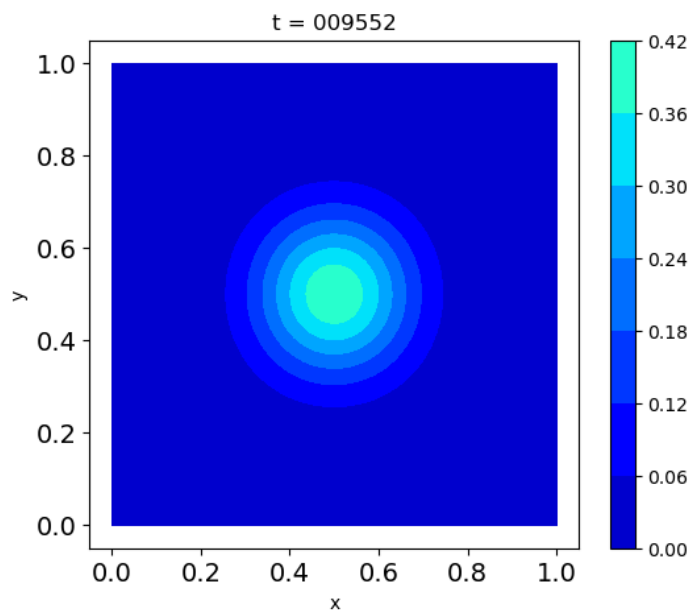
Execution Type	Jfun
Serial	5.752294971645e-03
p = 2	5.752294971645e-03
p = 4	5.752294971645e-03
p = 8	5.752294971645e-03

The error remains the same for different numbers of threads because the computation is deterministic and does not depend on the order of execution.

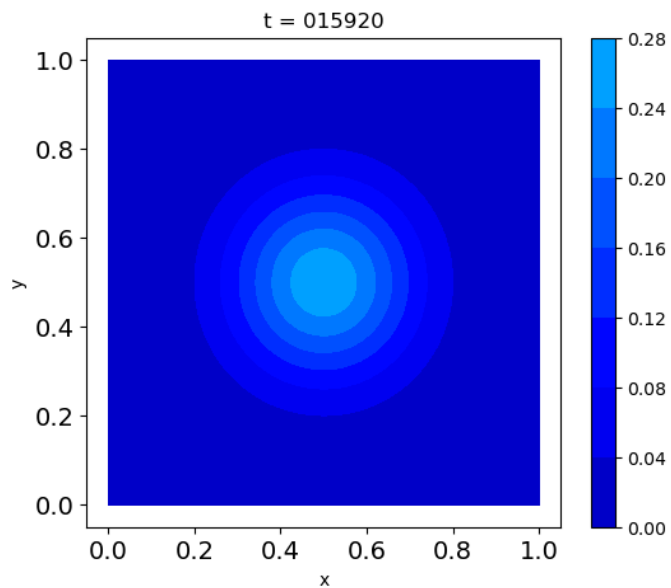
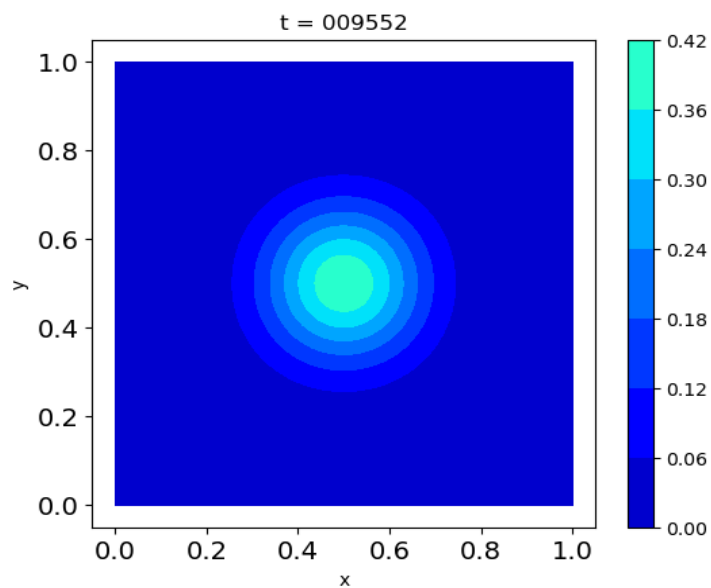
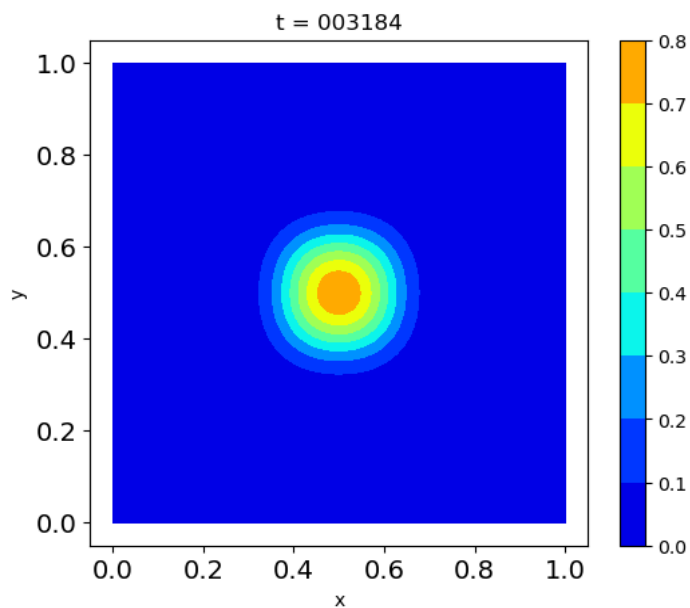
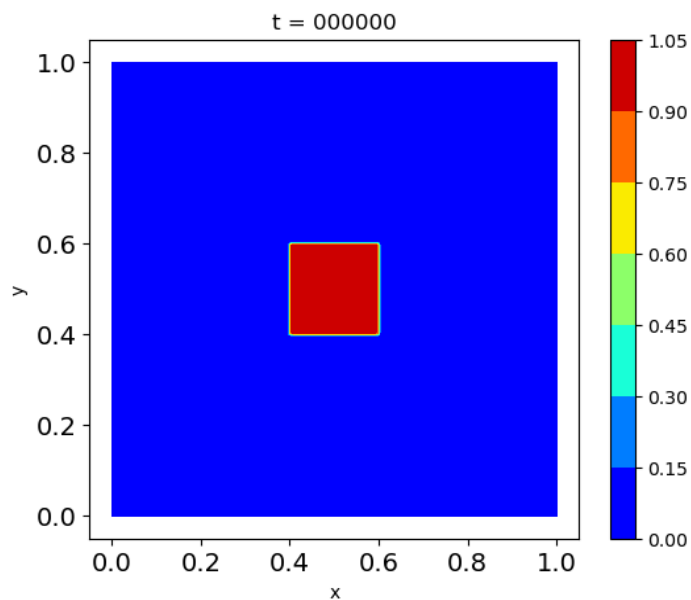
Q3)

Serial:

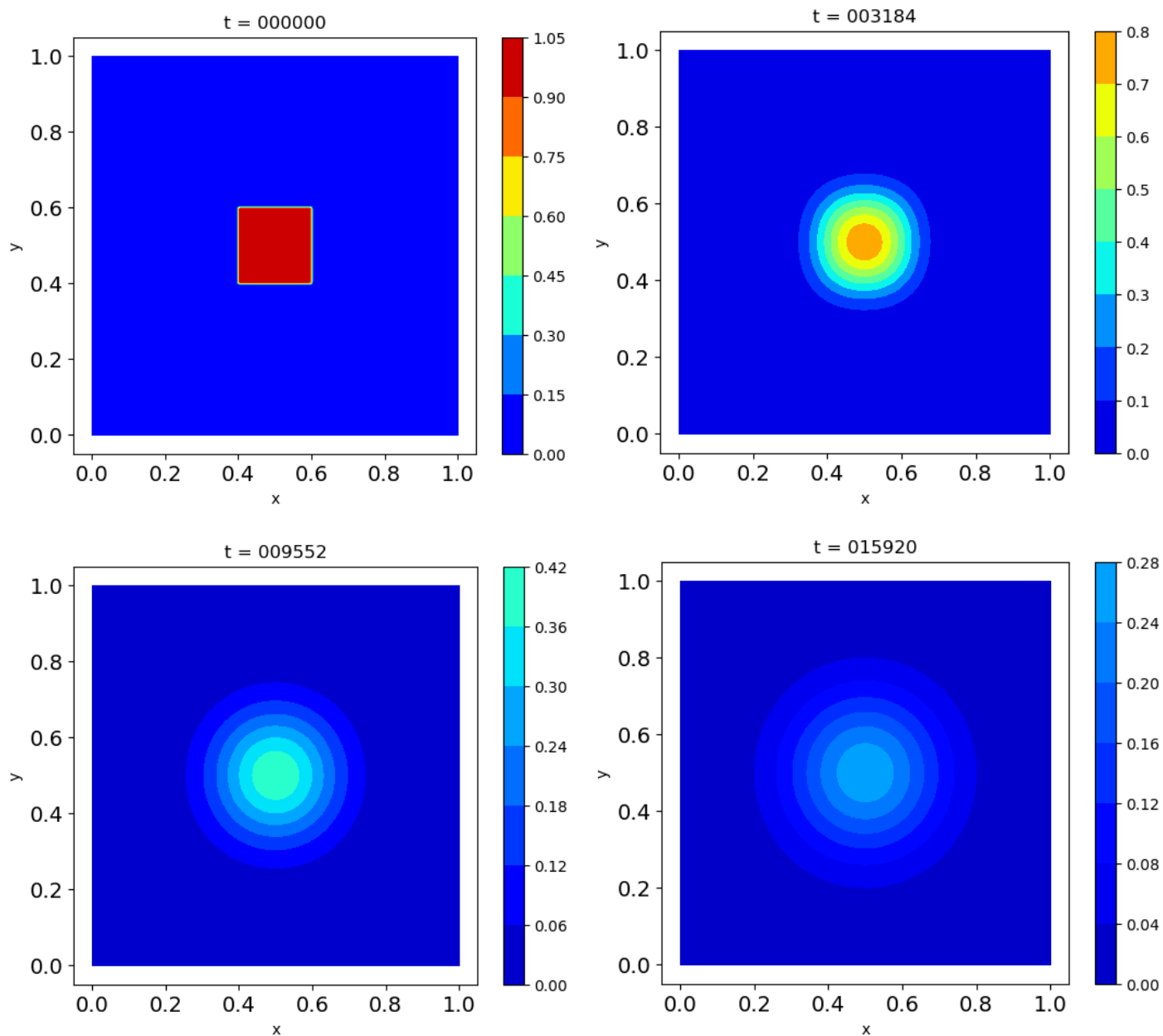




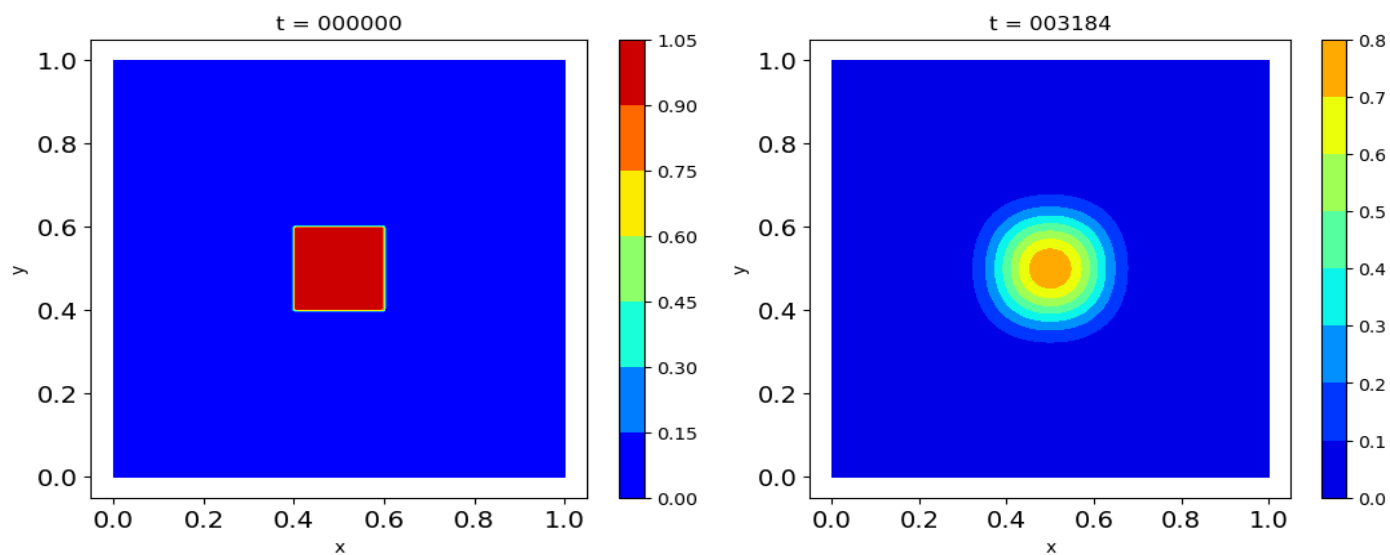
Parallel, p = 2:

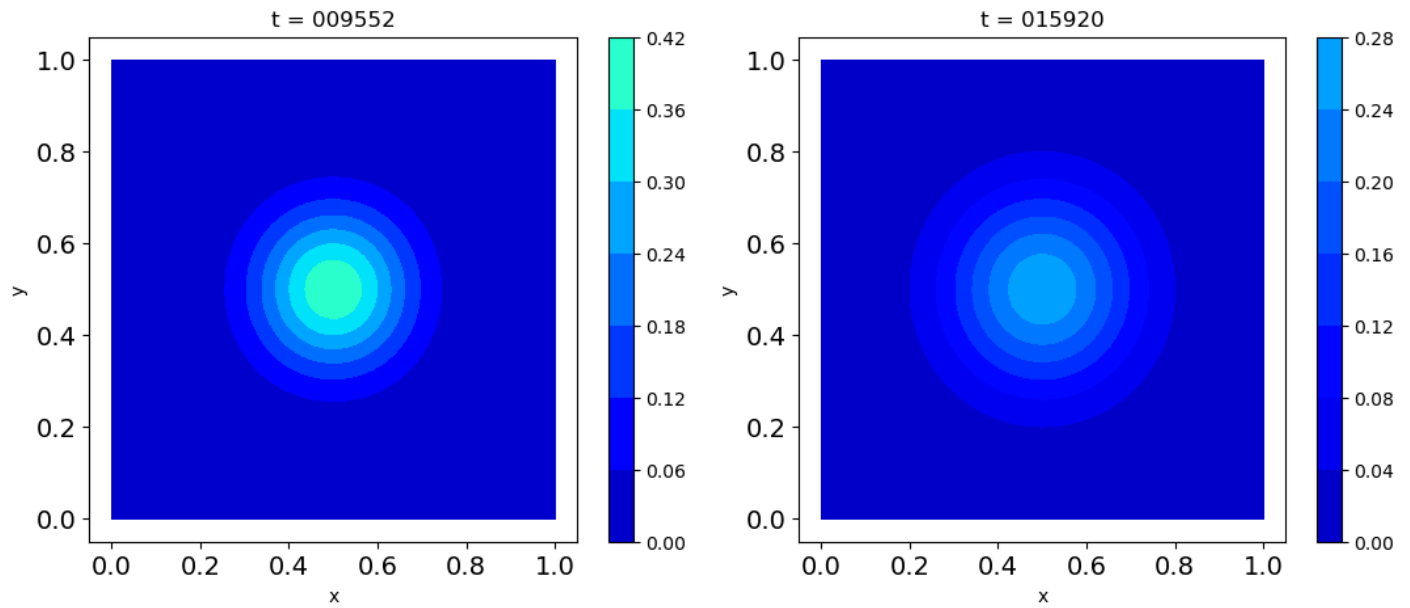


p = 4:



p = 8:





Final solution at some random points:

Point	Serial	p = 2	p = 4	p = 8
289, 71	0.009801	0.009801	0.009801	0.009801
157, 126	0.101792	0.101792	0.101792	0.101792
363, 228	0.006515	0.006515	0.006515	0.006515

From the table, it can be seen that the solution remains the same for different numbers of threads because the computation is deterministic and does not depend on the order of execution.